

MAE 263F – Homework 3

Yuchen Wang (Ph.D. in Mechanical Engineering)

University of California, Los Angeles

November 6, 2025

I. METHOD & ALGORITHMS

Question: A clear step-by-step description of your method, including pseudocode/algorithms for: (i) force/energy evaluation, (ii) time stepping, (iii) enforcement of Dirichlet constraints, and (iv) the path-planning or control law mapping the tracking error of node j to $\{x_c, y_c, \theta_c\}$.

Answer:

Throughout, the beam is discretized into N nodes with uniform spacing $\Delta L = L/(N-1)$. The generalized coordinate vector is

$$\mathbf{q} = [x_1 \ y_1 \ x_2 \ y_2 \ \dots \ x_N \ y_N]^T \in \mathbb{R}^{2N},$$

and the corresponding velocity is $\dot{\mathbf{q}}$. The lumped mass matrix $M \in \mathbb{R}^{2N \times 2N}$ is diagonal, with the same lumped mass m_{node} on the x and y DOFs of each node.

The “middle” tracking node is chosen as

$$j = \frac{N+1}{2},$$

so that (x_j, y_j) is unique for odd N .

A. (i) Force and energy evaluation

The total potential energy of the beam is written as

$$\Psi(\mathbf{q}) = \Psi_s(\mathbf{q}) + \Psi_b(\mathbf{q}) + \Psi_g(\mathbf{q}),$$

where Ψ_s is the stretching energy, Ψ_b is the bending energy, and Ψ_g is the gravitational potential energy.

a) Stretching energy.: We model each edge $(k, k+1)$ as a linear axial spring with rest length ΔL and stiffness $k_s = EA/\Delta L$, where E is Young’s modulus and A the cross-sectional area. The current segment length is

$$\ell_k(\mathbf{q}) = \left\| \begin{bmatrix} x_{k+1} - x_k \\ y_{k+1} - y_k \end{bmatrix} \right\|_2, \quad k = 1, \dots, N-1.$$

The stretching energy is

$$\Psi_s(\mathbf{q}) = \frac{1}{2} \sum_{k=1}^{N-1} k_s (\ell_k(\mathbf{q}) - \Delta L)^2.$$

b) Bending energy.: Bending is modeled by penalizing changes in the turning angle between adjacent segments $(k-1, k)$ and $(k, k+1)$. Let the unit tangent vector of segment k be

$$\mathbf{t}_k = \frac{1}{\ell_k} \begin{bmatrix} x_{k+1} - x_k \\ y_{k+1} - y_k \end{bmatrix}, \quad k = 1, \dots, N-1.$$

The discrete turning angle at node k is $\theta_k(\mathbf{q})$. The bending energy is

$$\Psi_b(\mathbf{q}) = \frac{1}{2} \sum_{k=2}^{N-1} k_b \theta_k(\mathbf{q})^2.$$

In the implementation, the stretching and bending forces and their Jacobians are assembled by two helper functions `getFs(q, EA, L)` and `getFb(q, EI, L)`:

$$\mathbf{f}_s(\mathbf{q}) = -\frac{\partial \Psi_s}{\partial \mathbf{q}}, \quad K_s(\mathbf{q}) = \frac{\partial \mathbf{f}_s}{\partial \mathbf{q}}, \quad (1)$$

$$\mathbf{f}_b(\mathbf{q}) = -\frac{\partial \Psi_b}{\partial \mathbf{q}}, \quad K_b(\mathbf{q}) = \frac{\partial \mathbf{f}_b}{\partial \mathbf{q}}. \quad (2)$$

c) Gravity.: Gravity acts on each node with magnitude $m_{\text{node}}g$ in the negative y direction. The gravitational potential energy is

$$\Psi_g(\mathbf{q}) = \sum_{k=1}^N m_{\text{node}} g y_k,$$

so that the corresponding nodal force is

$$\mathbf{f}_g = -\frac{\partial \Psi_g}{\partial \mathbf{q}} = [0 \ -m_{\text{node}}g \ 0 \ -m_{\text{node}}g \ \dots \ 0 \ -m_{\text{node}}g]^T.$$

d) Damping.: We add linear viscous damping

$$\mathbf{f}_v = -C\dot{\mathbf{q}},$$

with $C = \text{visc} \cdot I$ for a user-chosen scalar parameter `visc`.

e) Total internal force and stiffness.: The total internal force and tangent stiffness used in the Newton solver are

$$\mathbf{f}_{\text{int}}(\mathbf{q}, \dot{\mathbf{q}}) = \mathbf{f}_s(\mathbf{q}) + \mathbf{f}_b(\mathbf{q}) + \mathbf{f}_v(\dot{\mathbf{q}}), \quad (3)$$

$$K(\mathbf{q}) = K_s(\mathbf{q}) + K_b(\mathbf{q}). \quad (4)$$

Pseudocode: Force / energy evaluation

- 1) Given $\mathbf{q}$, compute segment lengths ℓ_k and tangents $\mathbf{t}_k$.
- 2) Evaluate stretching energy Ψ_s and force $\mathbf{f}_s$.
- 3) Evaluate bending energy Ψ_b and force $\mathbf{f}_b$.
- 4) Form gravity force $\mathbf{f}_g$ (constant in time).
- 5) For a given velocity $\dot{\mathbf{q}}$, compute damping force $\mathbf{f}_v = -C\dot{\mathbf{q}}$.
- 6) Return total internal force $\mathbf{f}_{\text{int}} = \mathbf{f}_s + \mathbf{f}_b + \mathbf{f}_v$ and, if needed, the tangent stiffness $K = K_s + K_b$.

B. (ii) Time stepping

Let $t_n = n\Delta t$ denote the discrete times. At each step, the previous position and velocity $(\mathbf{q}^n, \mathbf{u}^n)$ and seek the new position $\mathbf{q}^{n+1}$. I use a first-order implicit (backward Euler-type) integrator for the second-order system

$$M\ddot{\mathbf{q}} + \mathbf{f}_{\text{int}}(\mathbf{q}, \dot{\mathbf{q}}) = \mathbf{f}_g.$$

I approximate the new velocity as

$$\mathbf{u}^{n+1} \approx \frac{\mathbf{q}^{n+1} - \mathbf{q}^n}{\Delta t},$$

and define the residual of the discrete momentum balance at t_{n+1} as

$$\mathbf{r}(\mathbf{q}^{n+1}) = \frac{M}{\Delta t} \left(\frac{\mathbf{q}^{n+1} - \mathbf{q}^n}{\Delta t} - \mathbf{u}^n \right) - \mathbf{f}_{\text{int}}(\mathbf{q}^{n+1}, \mathbf{u}^{n+1}) - \mathbf{f}_g.$$

I then solve $\mathbf{r}(\mathbf{q}^{n+1}) = \mathbf{0}$ by Newton's method. The consistent Jacobian at iteration k is

$$J(\mathbf{q}_{(k)}^{n+1}) = \frac{\partial \mathbf{r}}{\partial \mathbf{q}} = \frac{M}{\Delta t^2} - K(\mathbf{q}_{(k)}^{n+1}) - \frac{C}{\Delta t},$$

where K is the tangent stiffness from the elastic forces and $C/\Delta t$ comes from differentiating the damping term.

Pseudocode: Implicit time step at t_{n+1}

- 1) Given $(\mathbf{q}^n, \mathbf{u}^n)$, form an initial guess $\mathbf{q}_{(0)}^{n+1} = \mathbf{q}^n$.
- 2) **For** $k = 0, 1, \dots$, until convergence:
 - a) Enforce Dirichlet constraints on $\mathbf{q}_{(k)}^{n+1}$.
 - b) Compute velocity $\mathbf{u}_{(k)}^{n+1} = (\mathbf{q}_{(k)}^{n+1} - \mathbf{q}^n)/\Delta t$.
 - c) Evaluate internal forces $\mathbf{f}_{\text{int}}(\mathbf{q}_{(k)}^{n+1}, \mathbf{u}_{(k)}^{n+1})$.
 - d) Assemble residual
$$\mathbf{r}_{(k)} = \frac{M}{\Delta t} \left(\frac{\mathbf{q}_{(k)}^{n+1} - \mathbf{q}^n}{\Delta t} - \mathbf{u}^n \right) - \mathbf{f}_{\text{int}}(\mathbf{q}_{(k)}^{n+1}, \mathbf{u}_{(k)}^{n+1}) - \mathbf{f}_g.$$
 - e) Assemble Jacobian $J_{(k)} = M/\Delta t^2 - K(\mathbf{q}_{(k)}^{n+1}) - C/\Delta t$.
 - f) Restrict $\mathbf{r}_{(k)}$ and $J_{(k)}$ to the free DOFs.
 - g) Solve $J_{(k)}^{\text{free}} \Delta \mathbf{q}^{\text{free}} = \mathbf{r}_{(k)}^{\text{free}}$ and update $\mathbf{q}_{(k+1)}^{n+1} = \mathbf{q}_{(k)}^{n+1} - \Delta \mathbf{q}$.
 - h) Check $\|\mathbf{r}_{(k)}^{\text{free}}\|$ against a tolerance.
- 3) Set $\mathbf{q}^{n+1} = \mathbf{q}_{(\text{final})}^{n+1}$ and update the velocity $\mathbf{u}^{n+1} = (\mathbf{q}^{n+1} - \mathbf{q}^n)/\Delta t$.

C. (iii) Enforcement of Dirichlet constraints

The beam has three groups of Dirichlet boundary conditions:

- The left end (node 1) is clamped at the origin,

$$x_1(t) = 0, \quad y_1(t) = 0.$$

- The right end (node N) is directly controlled by the robot,

$$x_N(t) = x_c(t), \quad y_N(t) = y_c(t).$$

- The penultimate node (node $N-1$) is constrained to lie one segment length ΔL back along the commanded angle $\theta_c(t)$:

$$x_{N-1}(t) = x_c(t) - \Delta L \cos \theta_c(t), \quad y_{N-1}(t) = y_c(t) - \Delta L \sin \theta_c(t).$$

Let $\mathcal{J}_{\text{fix}}$ denote the indices of all constrained DOFs in $\mathbf{q}$ (i.e. $(x_1, y_1, x_{N-1}, y_{N-1}, x_N, y_N)$), and let $\mathcal{J}_{\text{free}}$ be the complement set. In the implementation we explicitly construct two index arrays `fixed_index` and `free_index`.

At each time step, after computing the new control command $(x_c^{n+1}, y_c^{n+1}, \theta_c^{n+1})$, I build a vector $\mathbf{q}_{\text{fixed}}^{n+1}$ that contains the prescribed values on $\mathcal{J}_{\text{fix}}$:

$$\begin{aligned} x_1^{n+1} &= 0, & y_1^{n+1} &= 0, \\ x_N^{n+1} &= x_c^{n+1}, & y_N^{n+1} &= y_c^{n+1}, \\ x_{N-1}^{n+1} &= x_c^{n+1} - \Delta L \cos \theta_c^{n+1}, & y_{N-1}^{n+1} &= y_c^{n+1} - \Delta L \sin \theta_c^{n+1}. \end{aligned}$$

Pseudocode: Dirichlet enforcement inside Newton

- 1) Before each Newton iteration, enforce

$$\mathbf{q}_{(k)}^{n+1}[\mathcal{J}_{\text{fix}}] = \mathbf{q}_{\text{fixed}}^{n+1}[\mathcal{J}_{\text{fix}}].$$

- 2) When assembling the residual $\mathbf{r}_{(k)}$ and Jacobian $J_{(k)}$, extract the subvectors/matrices corresponding to $\mathcal{J}_{\text{free}}$:

$$\mathbf{r}_{(k)}^{\text{free}} = \mathbf{r}_{(k)}|_{\mathcal{J}_{\text{free}}}, \quad J_{(k)}^{\text{free}} = J_{(k)}|_{\mathcal{J}_{\text{free}}, \mathcal{J}_{\text{free}}}.$$

- 3) Solve for updates only on the free DOFs, and keep the fixed DOFs clamped to their prescribed values.

D. (iv) Control law for

$$\{x_c, y_c, \theta_c\}$$

The tracking objective is to drive the middle node j along the prescribed trajectory

$$x_j^*(t) = \frac{L}{2} \cos\left(\frac{\pi}{2} \frac{t}{1000}\right), \quad y_j^*(t) = -\frac{L}{2} \sin\left(\frac{\pi}{2} \frac{t}{1000}\right), \quad t \in [0, 1000] \text{ s.}$$

At a given time step we know the current configuration $\mathbf{q}^n$, so the current middle-node position is

$$x_j = x_j(\mathbf{q}^n), \quad y_j = y_j(\mathbf{q}^n).$$

I define the tracking error

$$e_x = x_j^* - x_j, \quad e_y = y_j^* - y_j.$$

The control law proceeds in three steps:

- 1) **Geometric guess.** Assuming the beam is roughly straight from $(0,0)$ to (x_c, y_c) and the middle node lies at half the length, I choose a geometric end-point guess

$$x_{\text{geom}} = 2x_j^*, \quad y_{\text{geom}} = 2y_j^*.$$

- 2) **Proportional position feedback.** I correct the geometric guess by a proportional term in the tracking error:

$$x_c^{\text{raw}} = x_{\text{geom}} + K_p e_x, \quad y_c^{\text{raw}} = y_{\text{geom}} + K_p e_y,$$

where $K_p > 0$ is a scalar gain.

In the implementation, this logic is encapsulated in a function `path_planner(...)`, which takes the current middle-node position, the desired middle-node position, and the previous command, and returns the new command (x_c, y_c, θ_c) under saturation.

Pseudocode: Control law

- 1) Input: x_j, y_j (current middle node), x_j^*, y_j^* (desired middle node), beam length L , gain K_p , limits $r_{\text{max}}, \theta_{\text{max}}$.

- 2) Compute tracking error $e_x = x_j^* - x_j$, $e_y = y_j^* - y_j$.
- 3) Geometric guess: $x_{\text{geom}} = 2x_j^*$, $y_{\text{geom}} = 2y_j^*$.
- 4) Proportional correction:

$$x_c^{\text{raw}} = x_{\text{geom}} + K_p e_x, \quad y_c^{\text{raw}} = y_{\text{geom}} + K_p e_y.$$

- 5) Output (x_c, y_c, θ_c) .

II. CONTROL INPUTS OVER TIME

Question: Plots of the control inputs $x_c(t)$, $y_c(t)$, $\theta_c(t)$ over time.

Answer:

Here are plots of the control inputs $x_c(t)$, $y_c(t)$, $\theta_c(t)$ over time.

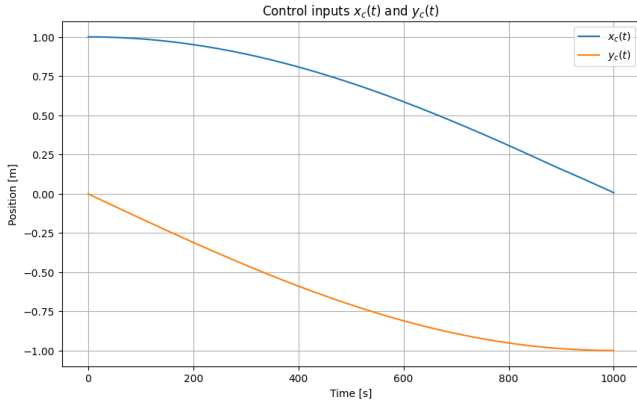


Fig. 1. plots of the control inputs $x_c(t)$, $y_c(t)$ over time

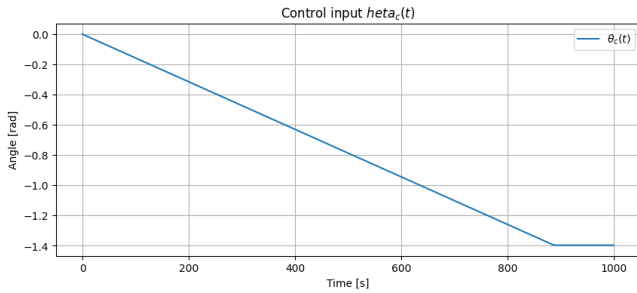


Fig. 2. plot of the control inputs $\theta_c(t)$ over time

III. BEAM SHAPE SNAPSHOTS

Question: At least five snapshots of the beam shape (node positions) during the motion.

Answer:

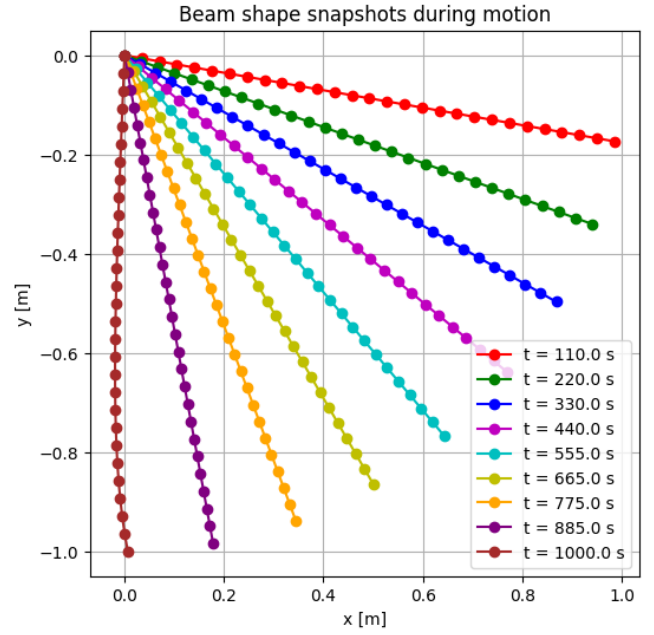


Fig. 3. plot of 9 snapshots of the beam shape

IV. FEASIBILITY & LIMITS

Question: A brief discussion of feasibility limits due to the robot's workspace and joint limits; explain how you handle infeasible commands (e.g., saturation or trajectory re-timing).

Answer: In practice, the end-effector (right end of the beam) is subject to geometric and mechanical limits that constrain the admissible commands $\{x_c, y_c, \theta_c\}$.

a) *Workspace limits:* The reachable region of the robot defines a bounded workspace

$$(x_c, y_c) \in \mathcal{W} = \{(x, y) \mid x_{\min} \leq x \leq x_{\max}, y_{\min} \leq y \leq y_{\max}\}.$$

If the desired command lies outside $\mathcal{W}$, it is projected back to the nearest feasible point on the boundary:

$$(x_c, y_c) \leftarrow \text{Proj}_{\mathcal{W}}(x_c, y_c).$$

b) *Joint and angular limits:* The rotation of the end-effector is bounded by

$$\theta_{\min} \leq \theta_c \leq \theta_{\max},$$

and its rate of change is limited to

$$|\dot{\theta}_c| \leq \omega_{\max}.$$

Violating these limits could correspond to physical self-collision or actuator saturation.

c) *Saturation handling:* We apply rate limiting to each component of the control increment $\Delta c = [\Delta x_c, \Delta y_c, \Delta \theta_c]^T$:

$$\Delta c \leftarrow \text{clip}(\Delta c; |\Delta c_i| \leq \Delta c_{\max}),$$

so the robot moves only within feasible velocity bounds even when the control demand is large.

d) *Trajectory re-timing*: If the planned trajectory requires motion beyond rate limits, the timing is uniformly scaled to maintain the same geometric path while reducing speed:

$$t' = \alpha t, \quad \alpha = \frac{\|\Delta c\|_{\text{actual}}}{\|\Delta c\|_{\text{demand}}} < 1.$$

This ensures smooth, feasible motion without overshoot.

By combining geometric projection, saturation, and trajectory re-timing, the controller guarantees that the commanded $\{x_c, y_c, \theta_c\}$ remain inside the physical workspace and within joint constraints, while preserving stability and continuity of motion.

CODE AND SOURCE ATTRIBUTION

The source code is adapted from *Simulation of a Beam with Fixed DOFs* and has been refined and formatted with the assistance of ChatGPT and Gemini.